

4장 공간단위 예측, 단위·피쳐·검증·해석

- 이 장에 나오는 Moran's I, 검증 점수, SHAP 값은 모두 `lecture_practice/chapter4/` 의 코드를 실제로 실행해 얻은 값
- 계산 구조를 보여 주기 위한 숫자에는 "예:"를 붙여 실측값과 구분함. 예시를 위해 지어낸 실행 결과는 없음

1. 이 장의 핵심 질문

□ 이 장은 세 가지 질문에 답함

○ **질문 1** — 격자 한 칸, 행정동 하나를 표의 한 행으로 만들려면 무엇부터 정해야 하고, "옆 동네 값" 같은 공간 피쳐는 어떻게 열이 되는가?

○ **질문 2** — 가까운 곳끼리 값이 닮는 자료에서 "무작위로 나눠 검증했다"는 왜 점수를 부풀리는가? 부풀지 않게 재는 방법은 무엇인가?

○ **질문 3** — 예측 점수와 순위표가 나왔을 때, 그것을 어디까지 믿고 어떻게 읽어야 하는가?

○ 이 장이 다루는 분석은 "어느 지역이 취약한가", "어느 지역에 시설이 부족한가"처럼 **공간단위마다 점수를 매기고 줄 세우는 예측임**

○ 이 구조는 8장 이후의 정책 장과 14장 이후의 비즈니스 장에서 계속 다시 나옴. 여기서 절차를 한 번 제대로 세우면 뒤의 장들은 같은 틀의 반복임

2. 같은 모델, 세 개의 점수

○ 이 장의 실습 자료(포인트 500개)에 같은 모델을 적합하고, **검증 방법만 바꿔** 성능을 재면 이렇게 나옴

검증 방법	평균 R^2
무작위로 나눠 검증	0.869
공간 블록으로 나눠 검증	0.798
공간 군집으로 나눠 검증	0.349

□ 자료도 모델도 그대로인데 점수가 0.869에서 0.349까지 내려감

○ 셋 중 어느 것이 "진짜 실력"인가 — 답은 **무엇을 재려 하는가**에 달려 있음. 관측이 있는 지역 사이를 메우는 능력이면 위쪽 값이, 학습한 적 없는 지역을 맞히는 능력이면 아래쪽 값이 그 능력에 가까움

○ 문제는 많은 분석이 아래쪽 능력을 주장하면서 위쪽 값을 보고한다는 것

○ 실제 연구에서도 같은 일이 확인됐음. Ploton 외(2020)는 아프리카 열대림 바이오매스 지도에서, 무작위 교차검증으로 높게 측정되던 모델 성능이 공간 교차검증에서 급락함을 보고했음. 발표된 생태 지도의 상당수가 검증되지 않은 외삽이었다는 지적임

○ 이 장 전체가 이 표 하나를 이해하는 과정임. 왜 벌어지는지(6절), 어떻게 재야 하는지(7절), 그 뒤에 결과를 어떻게 읽는지(8절)를 순서대로 다룸

3. 공간단위 예측의 여덟 단계

□ 공간단위 예측은 여덟 단계로 진행함. 앞 단계의 산출물이 뒷 단계의 입력이므로, 순서를 건너뛰면 뒷 단계의 판단 근거가 사라짐

○ 특히 검증 설계(6단계)를 정하지 않은 채 모델의 세부 설정(5단계)을 먼저 고르면, 그 설정은 부풀려진 점수에 맞춰 고른 것이 되고 나중에 되돌릴 수 없음

[1] 분석 단위·집계 규칙 정의	→ 단위 결정서, 집계 가중치
↓	
[2] 관측자료의 표 변환	→ 피처 행렬 X , 타깃 y
↓	
[3] 공간 피처 설계	→ 공간가중행렬 W , 공간 시차, 가용성 점검표
↓	
[4] 공간 구조 진단	→ 타깃의 Moran's I , 순열 검정 결과
↓	
[5] 모델 선택·적합	→ 기준모형 + 후보모형
↓	
[6] 공간 교차검증	→ 성능표, 폴드 간 표준편차
↓	
[7] 결과 해석	→ 피처 기여도, 잔차 진단
↓	
[8] 산출표·순위표	→ 단위별 예측표, 집계 순위표

그림 4.1 공간단위 예측의 여덟 단계와 단계별 산출물

○ 각 단계가 이 장의 어느 절에서 다뤄지는지는 다음과 같음

단계	무엇을 하는가	이 장의 절
1 분석 단위	표의 한 행을 무엇으로 돌지 정함	4절
2~3 표 변환·피쳐	관측치×피쳐 행렬을 만들고 공간 피쳐를 더함	5절
4 공간 진단	타깃이 공간적으로 뭉쳐 있는지 확인함	6절
6 공간 검증	검증을 어떻게 나눌지 정함	7절
5, 7~8 모델·해석·산출물	모델을 학습하고 결과를 읽음	8절

□ 5절부터 8절까지는 설명과 실습을 나누지 않음 — 각 절 첫머리에 실행할 명령이 나오고 그 아래에 코드와 실제 실행 결과가 이어짐. 순서대로 따라가면 이 장의 모든 수치를 직접 만들게 됨

□ 먼저 패키지를 설치함. GPU는 필요 없고 노트북에서 끝까지 돌아감

```
cd geoai_public # 저장소 루트 - 이 장의 명령은 모두 여기서 실행함
pip install -r lecture_practice/requirements-student.txt
```

○ 4장이 쓰는 패키지만 따로 보려면 `lecture_practice/chapter4/code/requirements.txt` 에 적혀 있으나, 설치하는 위 공통 파일 하나로 끝남

□ 이 장에서 실행할 코드는 네 개이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절
1	4-0-simdata-prep.py	실습 자료 생성(건물 200개·포인트 500개·격자 400개)	5
2	4-1-spatial-features.py	건물에서 형태·거리·위상 피쳐 추출	5.1
3	4-2-spatial-cv.py	Moran's I와 검증 사다리	6~7
4	4-5-grid-vulnerability-shap.py	격자 취약성 예측·SHAP·순위표	8

○ 2·3·4번은 1번이 만든 `data/` 폴더의 파일을 입력으로 받으므로 **1번을 먼저 실행**해야 함 — 없으면 "데이터가 없습니다"를 찍고 멈춤

○ 네 코드 모두 인터넷도 GPU도 쓰지 않음 — 자료를 내려받지 않고 코드가 직접 만들기 때문임

○ 본문에 인용한 출력은 모두 `lecture_practice/chapter4/results/` 의 실행 로그에서 그대로 가져온 것이며, 어느 파일에서 왔는지를 출력 블록 첫 줄에 적어 두었음

4. 표의 한 행 정하기: 분석 단위와 집계

4.1 다섯 가지 후보

- 분석에 들어가는 자료는 결국 표 하나로 정리되고, 그 표의 한 줄이 무엇인지가 첫 결정임
- 이 결정에 따라 어떤 자료를 모을 수 있는지, 어떤 피처를 만들 수 있는지, 결과를 누구에게 어떤 단위로 전달할지가 모두 달라짐

단위	쓰임	특징
포인트	사건·시설 위치	원자료, 집계 전 단계
격자	취약성·위험 비교	균일 단위, 행정경계와 무관
행정구역	예산·정책 집행	통계 공표·집행 단위
필지·건물	안전·에너지·과세	미시 단위
네트워크 권역	접근성·서비스	이동시간 기반 기능 단위

- **포인트**는 화재 신고 지점, 병원 위치처럼 사건·시설이 발생한 자리 그 자체임. 대부분의 분석은 포인트를 어떤 면 단위로 모으는 데서 시작함
- **격자**는 국토를 같은 크기의 정사각형(예: 500m × 500m)으로 자른 단위임. 모든 칸의 면적이 같아 지역 간 비교에서 면적 차이를 통제할 필요가 없고, 행정 경계가 바뀌어도 단위는 그대로임
- **행정구역**은 통계가 공표되고 예산이 집행되는 단위임. 분석 결과가 정책으로 이어지려면 결국 이 단위로 번역됨
- **필지·건물**은 자산 하나하나를 판정하는 문제(건물 안전 진단, 에너지 효율 평가)에 씀
- **네트워크 권역**은 도보 15분 안에 닿는 범위처럼 도로망 위의 이동시간으로 정의되는 단위임

4.2 칸을 바꾸면 결과가 바뀐다 — MAUP

- 같은 현상이라도 지도를 어떤 크기·모양의 칸으로 잘라 세느냐에 따라 다른 결과가 나옴. 이 현상을 **가변적 공간단위 문제**(Modifiable Areal Unit Problem, MAUP)라 함
- **크기의 문제**: 고령인구 비율을 500m 격자로 집계하면 고령자가 밀집한 작은 마을이 별도의 칸으로 드러남. 같은 자료를 1km 격자로 합치면 그 마을이 주변의 젊은 인구와 한 칸에 섞여 평균이 내려가고, 취약 지점이 지도에서 사라짐
- **경계의 문제**: 같은 마을이 행정동 경계를 어디에 긋느냐에 따라 취약 동에 속하기도 하고 평균적인 동에 속하기도 함
- MAUP는 고칠 수 있는 오류가 아니라 공간 집계에 원래 따라오는 성질임. 면적이 있는 단위로 값을 합치는 순간 그 단위 안의 분산은 사라지고, 단위의 크기와 경계가 결과를 좌우하는 요인이 됨
- 그래서 대응은 문제를 없애는 것이 아니라 관리하는 것임 — 분석 단위와 보고 단위를 분리하고(4.3),

어떤 단위를 썼는지 결과에 명시함

4.3 현상 단위와 집행 단위

- 취약성은 격자에서 발생하지만 예산은 행정구역에 배정됨
- 분석 단위를 처음부터 행정구역으로 잡으면 구역 안의 취약 지점과 안정 지점이 평균에 섞여 신호가 사라지고, 반대로 격자 예측만 내놓으면 예산을 어느 기관에 줄지 정할 수 없음
- 그래서 **현상 단위에서 예측하고 집행 단위로 집계**하는 두 단계를 둬. 취약성은 격자에서 계산하고, 예산을 논의할 때 읍면동으로 다시 모음
- 집계는 가중평균임. (가중치 × 예측값)의 합을 가중치의 합으로 나눔
- 예: 어느 행정구역 안에 격자 3개가 있고, 취약성 예측값이 40, 60, 80이라 하자
- 모든 격자를 동등하게 두면(가중치 1, 1, 1): $(40+60+80)/3 = 60$
- 인구가 각각 100, 500, 100이면(가중치 100, 500, 100): $(40 \times 100 + 60 \times 500 + 80 \times 100)/700 \approx 58.6$
- 같은 예측값인데 답이 다름. **가중치를 명시하지 않은 구역 평균은 무엇을 평균한 값인지 알 수 없음**

주의. 순위표는 집계 단위를 바꾸면 순서가 바뀜. 500m 격자에서 상위였던 지점이 1km 격자에서는 주변과 평균되어 순위에서 사라질 수 있음. 순위표의 제목이나 각주에 집계 단위와 가중치를 함께 적어야 함

5. 공간 피쳐 만들기

- 첫 번째 코드를 실행함. 5절부터 8절까지가 모두 쓰는 실습 자료를 만드는 단계이며 한 번만 실행하면 됨

```
python lecture_practice/chapter4/code/4-0-simdata-prep.py
```

```
# 출처: lecture_practice/chapter4/results/4-0-simdata-prep.log
건물 200개 → buildings.geojson, 지하철역 3개 → stations.geojson
공간 포인트 500개 → spatial_points.geojson
격자 400개(20×20) → grid_vulnerability.parquet
```

- 세 자료의 쓰임이 각각 다름 — 건물 200개는 5.1의 피쳐 추출에, 포인트 500개는 6~7절의 자기상관·검증에, 격자 400개는 8절의 취약성 예측에 씬

5.1 관측치와 피쳐

- 이 장의 모델은 관측치가 행으로, 각 관측치를 설명하는 피쳐가 열로 놓인 표를 입력받음
- **관측치**는 분석의 대상이고, **피쳐**는 그 대상을 설명하기 위해 측정하거나 계산한 변수임. 예측 대상 (취약성 점수 등)은 별도의 타깃 열로 둬م
- 8절이 쓰는 격자 시뮬레이션 자료의 첫 네 칸이 그 예임(방금 실행한 4-0-simdata-prep.py 가 만든 파일)

격자	고령인구율	1인가구율	기초수급율	병원거리(m)	평균 NDVI	취약성
0	0.25	0.30	0.07	1,251	0.271	35.7
1	0.24	0.27	0.07	1,313	0.432	44.6
2	0.28	0.36	0.10	1,508	0.344	60.9
3	0.22	0.35	0.08	710	0.291	53.2

- 이 표의 열들이 어디서 왔는지가 곧 이 절의 내용임
 - 고령인구율·1인가구율·기초수급율 — 통계 자료를 격자에 붙인 값
 - 병원거리 — 격자 중심에서 최근접 병원까지의 **거리 피쳐**
 - 평균 NDVI — 위성영상에서 계산한 **존 통계** (5.2)
 - 여기에 이웃 격자의 값을 요약한 **공간 시차**(5.4)를 열로 더할 수 있음
- 두 번째 코드를 실행함. 건물 200개를 행으로 놓고 열 일곱 개를 만드는 과정이며, 위 표의 "병원거리"와 같은 종류의 거리 피쳐가 어떻게 계산되는지를 보여 줌

```
python lecture_practice/chapter4/code/4-1-spatial-features.py
```

```

# 주석에 붙인 출력의 출처: lecture_practice/chapter4/results/4-1-spatial-features.log
import geopandas as gpd
import numpy as np

# [1] 건물 필지를 불러오고 기하 유효성부터 확인함
gdf = gpd.read_file(BUILDINGS_PATH)
gdf["geometry"] = gdf.geometry.make_valid()
# 출력: 불러온 건물: 200개 (좌표계 EPSG:32652)
#       유효하지 않은 기하 객체: 0개

# [2] 형태 피처 - 면적·둘레와 그 둘로 만드는 조밀도·종횡비
gdf["area_m2"] = gdf.geometry.area
gdf["perimeter_m"] = gdf.geometry.length
gdf["compactness"] = 4 * np.pi * gdf["area_m2"] / (gdf["perimeter_m"] ** 2)
# 출력: 면적 범위: 167.0 ~ 3393.9 m²
#       조밀도 범위: 0.423 ~ 0.785
#       종횡비 범위: 1.00 ~ 3.87

# [3] 거리 피처 - 건물 중심에서 최근점 지하철역까지, 그리고 중심점까지
gdf["dist_to_station"] = gdf.geometry.apply(
    lambda g: nearest_station_dist(g, stations))
gdf["dist_to_center"] = gdf.geometry.centroid.distance(center)
# 출력: 지하철역 거리: 96 ~ 3415 m
#       중심 거리: 141 ~ 4064 m

# [4] 위상 피처 - 50m 버퍼 안에 걸리는 다른 건물의 수
gdf["neighbors_50m"] = neighbor_counts
# 출력: 50m 내 인접 건물 수: 0 ~ 2
#       평균 인접 건물 수: 0.1
#       총 7개 공간 피처 추출 완료

```

□ 화면에 나온 값에서 읽을 것

- 만든 열 일곱 개가 세 갈래로 갈림 — 도형 자체에서 나오는 형태 피처(면적·둘레·조밀도·종횡비), 다른 대상까지 재는 거리 피처(지하철역·중심점), 주변에 무엇이 몇 개 있는지 세는 위상 피처(인접 건물 수)
- 조밀도 최대값이 0.785 인 것은 계산이 맞게 돌아갔다는 확인임. 조밀도 = $4\pi \times \text{면적} \div \text{둘레}^2$ 이므로 정사각형이면 $\pi/4 = 0.785$ 가 되고, 이 자료의 건물은 회전한 직사각형이라 그보다 커질 수 없음
- 평균 인접 건물 수가 0.1 로 거의 0임 — 건물 200개를 6km×6km 범위에 흩뿌린 자료라 50m 안에 이웃이 잡히는 건물이 드물. 위상 피처는 밀집도가 실제로 있는 자료에서만 값이 갈림

5.2 존 통계: 영상을 표의 한 칸으로

□ 격자 경계는 벡터이고 위성영상은 픽셀 격자임. 두 자료를 연결하는 계산이 **존 통계**(zonal statistics)임

□ 세 단계로 진행함

1. 격자 경계와 영상의 좌표계를 일치시킴 (2장에서 배운 그 작업)
2. 경계 안에 들어오는 픽셀을 골라냄
3. 그 픽셀들의 통계량을 계산해 표의 한 칸에 적음

□ 어떤 통계량을 고르느냐가 피처의 의미를 정함. 평균은 구역의 전반적 수준을, 표준편차는 구역 내부가 균질한지를, 임계값 초과 면적 비율은 조건을 만족하는 부분이 얼마나 되는지를 점

○ 위 표의 "평균 NDVI" 열이 이 계산의 산출물임 — 격자 한 칸 안 픽셀들의 NDVI 평균. 영상 자체를 모델에 직접 넣는 방법은 5장에서 다룸

5.3 어느 칸이 이웃인가 — 공간가중행렬

□ "주변 격자의 고령인구율"을 새 열로 만들려면, 격자마다 어느 칸이 이웃인지부터 정해야 함

○ 이웃을 변으로 맞닿은 네 칸으로 볼지, 대각선을 포함한 여덟 칸으로 볼지, 반경 1km 안의 모든 칸으로 볼지에 따라 같은 자료에서 다른 값이 나옴

○ 이 정의를 담는 도구가 **공간가중행렬**(spatial weights matrix) W 임. n 개의 공간단위가 있으면 W 는 $n \times n$ 행렬이고, 이웃이 아니면 0, 이웃이면 양수를 담음

방식	정의	적합한 자료
rook 인접	변을 맞댄 4칸	격자
queen 인접	모서리를 포함한 8칸	격자, 대각으로도 이어지는 현상
거리 임계	반경 d 안의 모든 단위	점자료, 관측소망
k -최근접	가장 가까운 k 개	단위 밀도가 고르지 않은 자료

○ 행정구역처럼 크기가 제각각인 단위에서는 거리 임계가 문제를 일으킴. 임계를 작게 잡으면 넓은 농촌 구역이 이웃을 하나도 갖지 못하고, 크게 잡으면 좁은 도심 구역이 수십 개의 이웃을 가짐. k -최근접을 쓰면 모든 단위가 같은 수의 이웃을 가짐

□ 이웃을 정한 뒤에는 **행 표준화**를 함 — 각 행의 가중치를 그 행의 합으로 나눠 합이 1이 되게 만들

○ 이유: 이웃 수가 위치마다 다르기 때문임. 격자 내부 칸은 이웃이 넷이지만 모서리 칸은 둘뿐임. 표준화 없이 이웃 값을 더하면 이웃이 많은 위치일수록 합이 커져서, 주변의 조건이 아니라 **이웃 개수**를 반영하는 열이 만들어짐. 표준화하면 이웃 값의 가중평균이 되어 원변수와 같은 단위로 읽힘

```
from libpysal.weights import KNN
```

```
w = KNN.from_dataframe(gdf, k=8) # 각 지점의 최근접 8개를 이웃으로
```

```
w.transform = "r" # 행 표준화
```

○ 위 두 줄은 6~7절에서 실행하는 4-2-spatial-cv.py 가 이웃을 정하는 부분과 같음 — 포인트 500개에 k=8 이웃과 행 표준화를 적용한 결과가 6.2의 Moran's I임

5.4 공간 시차: 이웃 값의 가중평균

○ 변수 x의 **공간 시차**(spatial lag)는 W로 정의된 이웃 값의 가중평균임. 시계열에서 한 시점 전 값을 시차 변수로 쓰듯, 공간에서는 옆 칸들의 값을 시차 변수로 씀

□ 예: 행정동 i의 이웃 네 곳의 고령인구율이 0.18, 0.22, 0.31, 0.25이고 행 표준화로 각 이웃에 0.25씩 배정되면

○ 공간 시차 = $0.25 \times 0.18 + 0.25 \times 0.22 + 0.25 \times 0.31 + 0.25 \times 0.25 = 0.24$

○ 이 값은 동 i를 둘러싼 지역의 고령화 수준임

□ 원변수와 시차를 함께 보면 상대 위치가 드러남. 동 i 자체의 고령인구율이 0.32라면 주변보다 8%p 높은 국지적 고령 밀집지이고, 0.16이라면 고령화된 지역 안의 상대적으로 젊은 동임. 원변수 하나만으로는 두 경우를 구분할 수 없음

○ 그래서 원변수와 시차의 차(위 예에서 $0.32 - 0.24 = +0.08$)를 별도 열로 만들기도 함. 결정 트리의 분기는 한 열에 대한 임계값 비교라서, 두 열의 차이를 스스로 만들어 내지 못하기 때문임

5.5 만든 열을 써도 되는가 — 두 가지 함정

□ 만들 수 있는 열이 모두 쓸 수 있는 열은 아님. 판정 기준은 하나 — **예측을 수행하는 시점에 그 값을 실제로 받을 수 있는가**

○ 받을 수 없는 값이 열에 섞이는 현상을 **목표값 누수**(target leakage)라 함. 예측 대상이 올해의 화재 발생 여부인데, 올해 화재 진압에 투입된 인력을 열로 넣으면 여기에 해당함 — 그 값은 화재가 난 뒤에야 생기는 값임

주의. 목표값 누수는 오류 메시지 없이 지나감. 코드는 정상 종료하고 검증 점수는 오히려 올라가며, 기여도 표에서 그 열이 상위로 옴. 피처를 만들 때마다 "이 값을 예측 시점에 어디서 받아 오는가"를 한 줄로 적고, 답을 적을 수 없으면 후보에서 제외함

□ 좌표를 피처로 넣는 결정은 별도의 판정이 필요함

○ "이 격자가 지도의 왼쪽에 있다"는 사실은 취약성과 직접 관련이 없어 보이는데, 좌표를 열로 넣으면 무작위 검증 점수가 올라감. 이유는 다음과 같음

1. 실제 취약성은 표의 열만으로 결정되지 않음. 마을 공동체의 결속처럼 어느 열에도 기록되지 않았지만 지역에 따라 부드럽게 변하는 효과가 섞여 있음. 이를 **잠재 공간 구조**(latent spatial structure)라 함
2. 좌표가 주어지면 모델은 "좌표가 이 범위인 격자는 타깃이 대체로 높다"는 규칙을 학습함 — 잠재 구조를 위치로 보간하는 것
3. 무작위 검증에서는 검증 격자 바로 옆 격자가 학습에 남아 있어 이 보간이 통하지만, 공간 분리 검증에서는 검증 지역 전체가 빠지므로 보간의 근거가 사라지고 상승분도 사라짐

○ 좌표를 넣었다면 공간 분리 검증 결과를 반드시 함께 보고, 피쳐 기여도에서 좌표가 상위로 오르는지 확인함 — 8절에서 이 현상이 실제 수치로 나타남

6. 공간 자기상관 재기: Moran's I

□ 세 번째 코드를 실행함. 6절과 7절의 수치가 전부 이 한 번의 실행에서 나옴

```
python lecture_practice/chapter4/code/4-2-spatial-cv.py
```

○ 출력은 여덟 구간으로 나뉨 — 자료 확인(1)과 Moran's I(2)·LISA(3)가 6절, 피쳐 구성(4)과 세 가지 검증(5~7)이 7절, 시드 반복(8)이 7.3의 마지막 대목임

6.1 계산 구조

□ 가까운 위치의 값이 서로 닮는 성질을 **공간 자기상관**(spatial autocorrelation)이라 함. 2절의 점수 추락을 만든 성질이 바로 이것임

○ 그 정도를 재는 표준 지표가 **Moran's I**임(Moran, 1950). 양수이면 비슷한 값끼리 이웃해 있고, 0 근처이면 배열이 위치와 무관하며, 음수이면 높은 값과 낮은 값이 교차 배열되어 있다는 뜻

□ 작은 예로 계산을 따라가면 구조가 보임. 네 칸이 일렬로 놓여 있고 값이 10, 12, 4, 6이며, 좌우로 맞닿은 칸만 이웃이라 하자

1. 평균은 8, 각 칸의 편차는 +2, +4, -4, -2
2. 이웃한 두 칸의 편차를 곱해 다 더함(공간 교차곱): $2 \times [(+2)(+4) + (+4)(-4) + (-4)(-2)] = 2 \times (8 - 16 + 8) = 0$
3. 편차제곱합: $4 + 16 + 16 + 4 = 40$
4. $I = (n / \text{가중치합}) \times (\text{교차곱} / \text{편차제곱합}) = (4 / 6) \times (0 / 40) = 0$

○ 교차곱이 핵심임. 이웃끼리 둘 다 평균보다 높거나 둘 다 낮으면 곱이 양수, 한쪽만 높으면 음수가 됨. 그래서 I의 부호가 "닮은 값끼리 붙어 있는가"를 잴

○ 이 계산 예에는 주의할 조건이 하나 붙음. 높은 두 칸(10, 12)과 낮은 두 칸(4, 6)이 나뉘어 붙어 있는데도 전역 지표는 0임 — 두 덩어리의 경계에서 나온 음의 곱이 양의 곱을 상쇄했기 때문. **전역 Moran's I가 0에 가깝다고 공간 구조가 없다고 단정할 수 없음**

○ 유의성은 순열 검정으로 판정함. 값들의 자리를 무작위로 여러 번(예: 999회) 뒤섞어 I를 다시 계산하면 "자기상관이 없을 때 I가 얼마쯤 나오는지"의 분포를 얻고, 관측된 I가 그 분포의 극단에 있으면 우연으로 보기 어렵다고 판정함

6.2 실행 결과와 검증 설계 판정

□ 방금 실행한 코드의 앞 세 구간이 이 절의 답임 — 포인트 500개의 타겟에 k=8 이웃과 행 표준화를 적용한 결과

```
# 출처: lecture_practice/chapter4/results/4-2-spatial-cv.log [1~3 구간]

--- 1. 공간 포인트 관측 데이터 불러오기 ---

불러온 포인트: 500개 (좌표계 EPSG:32652)

타겟 범위: -7.2 ~ 72.8

타겟 평균: 30.8, 표준편차: 15.1

--- 2. Global Moran's I ---

Moran's I 값: 0.8660

기대값 E[I]: -0.0020

Z-score: 40.8371

p-value: 0.0010

해석: 강한 양의 공간 자기상관 → 공간 CV 필수

--- 3. Local Moran's I (LISA) ---

유의한 클러스터 분류 (p < 0.05):

HH (Hot Spot): 159개

LH (Low-High): 1개

LL (Cold Spot): 134개

HL (High-Low): 0개

Not Significant: 206개
```

□ 화면에 나온 값에서 읽을 것

○ Moran's I = **0.866**, 기대값 -0.002, z-score **40.84**, p = 0.001 — 무작위 배치에서 기대되는 값에서 표준오차의 40배 넘게 떨어져 있음. 이 자료의 값은 위치와 무관하게 배열되어 있지 않음

○ 3구간의 LISA는 전역 지표를 단위 하나하나로 쪼갠 값임. 높은 값끼리 뭉친 자리(HH) 159개와 낮은 값끼리 뭉친 자리(LL) 134개가 유의하게 나왔고, 높은 값이 낮은 값에 둘러싸인 자리(HL)는 0개임

○ 6.1에서 전역 지표만으로는 자리를 알 수 없다고 한 덩어리들이 여기서 개수와 종류로 나옴 — 전역 값과 국지 값을 함께 보고하는 이유임

○ 이 진단이 다음 절의 검증 설계를 정함

Moran's I	공간 자기상관	검증 설계
+0.7 ~ +1.0	강함	공간 분리 검증 필수
+0.3 ~ +0.7	중간	공간 분리 검증 권장, 무작위 검증과 병기
-0.3 ~ +0.3	약함	무작위 검증 사용 가능

○ 실습 자료의 0.866은 첫 행에 해당함 — 무작위 검증만으로는 안 되는 자료임

주의. Moran's I 값은 이웃을 어떻게 정하느냐에 따라 달라짐. k-최근접의 k나 거리 임계값을 바꾸면 같은 자료에서 다른 값이 나옴. 보고할 때 이웃 정의와 순열 검정 반복 수를 함께 적음

7. 공간 교차검증

□ 6절에서 실행한 그 코드의 뒤쪽 구간을 읽음. 건너뛰고 왔다면 여기서 실행함

```
python lecture_practice/chapter4/code/4-2-spatial-cv.py
```

```
# 출처: lecture_practice/chapter4/results/4-2-spatial-cv.log [5~7 구간]
```

```
--- 5. Random CV vs Block CV 비교 ---
```

```
Random K-Fold CV:
```

```
R² scores: [0.866, 0.876, 0.855, 0.870, 0.878]
```

```
평균 R²: 0.869 +/- 0.008
```

```
블록 수: 36개 (5km 격자)
```

```
Spatial Block CV:
```

```
R² scores: [0.878, 0.519, 0.828, 0.917, 0.849]
```

```
평균 R²: 0.798 +/- 0.143
```

```
--- 6. Cluster CV ---
```

```
Cluster CV:
```

```
R² scores: [-0.179, 0.539, 0.508, 0.371, 0.509]
```

```
평균 R²: 0.349 +/- 0.271
```

```
--- 7. CV 기법 비교 요약 ---
```

기법		평균 R²		표준편차		과대추정
-----		-----		-----		-----
Random CV		0.869		0.008		+0.071
Block CV		0.798		0.143		+0.000
Cluster CV		0.349		0.271		-0.449

○ 2절 첫머리의 표가 이 출력에서 나온 것임. 왜 이렇게 벌어지는지를 7.1에서, 무엇을 어떻게 나눈 결과인지를 7.2에서, 폴드별 값을 어떻게 읽는지를 7.3에서 봄

7.1 무작위 분할은 왜 점수를 부풀리는가

□ 자기상관이 강한 자료를 무작위로 학습·검증에 나누면, 검증 표본의 **바로 옆 표본이 학습에 들어감**

○ 값이 공간적으로 닮아 있으므로, 모델은 인접 관측을 보간하는 것만으로 검증 표본을 상당 부분 맞힐 수 있음

○ 이때 점수가 재는 것은 새 지역으로 일반화하는 능력이 아니라 **이웃값을 보간하는 능력**임. 두 능력을 구분하지 않으면, 학습 지역 안에서 켜 점수를 학습하지 않은 지역의 성능이라고 발표하게 됨 — 2절의 Ploton 사례가 정확히 이 상황임

○ 이 현상을 **공간 분할 누수**(spatial split leakage)라 함. 5.5의 목표값 누수와는 다른 문제임 — 목표값 누수는 열 구성의 문제라 열을 빼서 고치고, 공간 분할 누수는 검증 설계의 문제라 나누는 방식을 바꿔서 고침

7.2 공간을 나누는 네 가지 설계

○ 해법의 공통 원리는 하나 — 표본이 아니라 **지역을 통째로** 폴드에 배정함

설계	분할 방법	평가하는 상황
무작위 교차검증	행을 무작위로 분할	공간 구조가 없는 표본
블록 교차검증	좌표를 격자 블록으로 잘라 블록 단위 분할	인접하지 않은 블록의 예측
버퍼 교차검증	검증지 주변 학습표본을 제거	가까운 이웃 없이 예측
군집 교차검증	좌표를 군집으로 묶어 군집 단위 분할	학습하지 않은 권역으로 이전

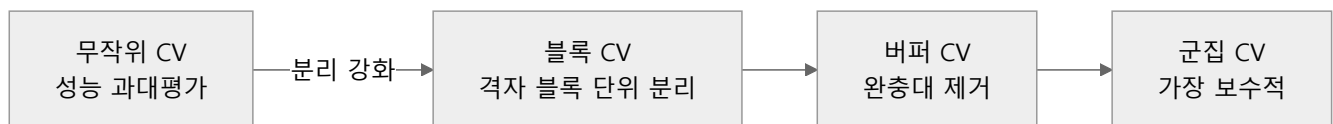


그림 4.2 검증 설계의 분리 강화 사다리

- 오른쪽으로 갈수록 학습과 검증의 공간 분리가 강해지고, 재는 성능은 "학습하지 않은 지역"의 성능에 가까워짐
- 어느 설계를 쓸지는 모델이 실제로 놓일 상황에 맞춤(Roberts et al., 2017). 같은 도시 안의 빈 격자를 메우는 용도면 블록 검증이, 다른 시군으로 모델을 이전할 계획이면 군집 검증이 그 상황에 대응함
- 구현은 짧음. 각 표본에 블록 식별자를 붙이고, 같은 블록이 같은 폴드에 가도록 `GroupKFold` 로 나눔

```

from sklearn.model_selection import GroupKFold, cross_val_score

# 좌표를 5km 격자로 잘라 블록 식별자 부여
gdf["block_id"] = ((gdf.geometry.x // 5000).astype(int) * 10000
                  + (gdf.geometry.y // 5000).astype(int))

# 같은 블록은 같은 폴드로 - 블록 단위 분리
scores = cross_val_score(model, X, y, scoring="r2",
                        cv=GroupKFold(n_splits=5), groups=gdf["block_id"])
  
```

- 위 코드가 첫머리 출력의 5구간을 만들었음 — 포인트 500개가 5km 블록 36 개로 나뉘고, 같은 블록은 통째로 한 폴드에 들어감

7.3 실행 결과 읽기: 검증 사다리

- 성능지표는 회귀에서 R^2 를 씀 — "모든 단위에 평균을 예측하는 규칙"과 비교해 모델이 설명한 변동의 비율. $R^2=0$ 이면 평균 예측과 같은 수준이고, **음수이면 평균 예측보다 나쁨**
- 2절에서 본 표를 이제 전체 열로 봄. 첫머리 출력의 5~7 구간을 옮긴 것이며, 폴드 최저는 6구간의 R^2 목록에 찍힌 값임

검증 설계	평균 R^2	폴드 간 표준편차	폴드 최저
무작위 5-폴드	0.869	0.008	—
블록 CV	0.798	0.143	—
군집 CV	0.349	0.271	-0.179

□ 세 가지를 순서대로 읽음

1. **분리가 강해질수록 평균이 내려감** — $0.869 \rightarrow 0.798 \rightarrow 0.349$. 무작위 검증과의 격차가 곧 인접 누수로 부풀었던 몫임
2. **분리가 강해질수록 폴드 간 변동이 커짐** — 표준편차 $0.008 \rightarrow 0.143 \rightarrow 0.271$. 무작위 검증은 다섯 폴드가 좁은 구간에 모여 있지만, 군집 검증은 어느 권역을 빼느냐에 따라 점수가 크게 출렁임
3. **폴드 최저 -0.179** — 군집 검증의 어느 한 권역에 대해서는 모델이 평균 예측보다 못했다는 뜻. 평균 0.349는 이런 폴드를 포함한 값이므로, 확장을 계획한다면 평균이 아니라 최저 폴드를 근거로 삼음

주의. 무작위와 블록의 차이는 0.071인데, 블록 검증의 폴드 간 표준편차는 0.143임. 재려는 차이가 측정의 흔들림보다 작음. 이런 값은 "차이가 있다"의 근거로 인용하지 않음 — 평균만 보고하고 표준편차를 빼면 이 판단 자체를 할 수 없으므로, 폴드 간 표준편차는 항상 함께 적음

□ 그 흔들림의 폭을 코드의 8구간이 직접 잴. 같은 생성식으로 포인트 500개를 다시 뽑아 사다리를 네 번 더 올림

출처: lecture_practice/chapter4/results/4-2-spatial-cv.log [8 구간 (C)]

(C) 생성 시드 교체 - 포인트를 다시 뽑는다 (주축)

시드	Random	Block	Cluster	R-B	R-C	폴드SD(B/C)
0	0.865	0.814	0.170	+0.052	-0.695	0.089/0.382
1	0.875	0.752	-0.872	+0.123	-1.747	0.223/1.403
2	0.863	0.766	-0.429	+0.098	-1.293	0.141/0.912
42	0.869	0.798	0.349	+0.071	-0.519	0.143/0.271

□ 화면에 나온 값에서 읽을 것

○ 무작위 검증은 시드를 바꿔도 0.863~0.875 로 거의 움직이지 않는데, 군집 검증은 0.349 에서 -0.872 까지 내려감 — 학습하지 않은 권역의 성능은 자료가 조금만 달라져도 크게 흔들림

○ 12번(4시드×3축)을 합치면 무작위-블록은 +0.031~+0.123, 무작위-군집은 -1.747~-0.517 임. 두 범위 모두 0을 품지 않으므로 **부호는 흔들리지 않고 크기만 흔들림**

○ 그래서 이 절이 인용한 0.071(무작위-블록)과 0.519(무작위-군집)는 "이 실행에서 이만큼"이지 "일반적으로 이만큼"이 아님. 과대추정의 크기를 인용할 때는 한 번의 실행이 아니라 이 범위를 적음

8. 격자 취약성 예측과 결과 해석

□ 네 번째 코드를 실행함. 8절의 수치가 전부 이 한 번의 실행에서 나옴

```
python lecture_practice/chapter4/code/4-5-grid-vulnerability-shap.py
```

○ 출력은 다섯 덩어리임 — 자기상관 진단(8.1), 세 가지 검증(8.2), SHAP 기여도와 잔차 진단(8.3), 우선순위 표(8.4), 그리고 시드를 바꿔 사다리를 다시 올린 반복 구간

8.1 자료와 기준모형

□ 자료는 가상 지역을 20×20으로 자른 격자 400개 시뮬레이션임. 피쳐는 5.1의 표에 나온 다섯 열에 대중교통 거리와 좌표(x, y)를 더한 8개 — 좌표를 **일부러** 넣어서 5.5의 현상을 관찰함

○ 타깃(취약성)의 Moran's I는 **0.817**(p=0.001) — 6.2의 판정 표에서 공간 분리 검증이 필수인 구간

```
# 출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log
```

분석 단위: 400개 격자(20×20), 피쳐 8개, 개인 식별 불가

```
[4.3 공간 자기상관] Global Moran's I = 0.817 (p = 0.001)
```

→ I>0.3, 유의 → 공간 자기상관 강함, 공간 CV 필수(아니면 누수)

○ 모델을 적합하기 전에 **기준모형**(baseline model)을 정함. R² 0.6이라는 숫자만으로는 그 모델이 쓸 만한지 답할 수 없음 — 평균 예측보다 크게 나은 것일 수도, 단순 선형회귀로도 되는 수준일 수도 있음. 그래서 전체 평균(R²=0 지점)과 선형회귀를 비교 대상으로 두고, 복잡한 모델의 성능은 항상 이 기준선과의 차이로 말함

8.2 랜덤 포레스트

□ 이 실습의 모델은 **랜덤 포레스트**(Random Forest)임(Breiman, 2001). 결정 트리 하나는 학습자료가 조금만 바뀌어도 모양이 크게 달라지는데, 이 불안정을 여러 트리의 평균으로 줄이는 방법임

1. 원자로 n행에서 중복을 허용해 n행을 다시 뽑음 (트리마다 다른 표본)
2. 각 분기에서 전체 피처를 다 보지 않고 무작위로 고른 일부에서만 최적 분기를 찾음 (트리마다 다른 피처 후보)
3. 트리를 깊게 키움
4. 1~3을 수백 회 반복하고, 회귀는 예측의 평균, 분류는 다수결로 결합함

- 2단계가 핵심임. 표본만 다시 뽑으면 강한 피처 하나가 모든 트리의 최상단을 차지해 트리들이 서로 닮게 되고, 닮은 예측은 평균해도 분산이 줄지 않음. 피처까지 무작위로 골라야 트리들이 실제로 달라짐
- 격자 자료에 적합한 뒤 세 가지 검증으로 잼. 코드는 비교를 위해 XGBoost도 같은 조건으로 함께 돌림

출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log

[4.5 공간 CV 비교] Random vs Block vs Cluster (R^2)

모델	Random	Block	Cluster	과대추정(R-B)
RF	0.639	0.535	0.373	+0.104
XGBoost	0.629	0.490	0.183	+0.139

→ Random CV가 Block/Cluster보다 높다 = 공간 누수로 인한 과대추정.

□ 화면에 나온 값에서 읽을 것

- 7.3의 사다리가 여기서도 반복됨 — 분리가 강해질수록 점수가 내려감. 무작위 검증의 0.639 를 "학습하지 않은 지역"의 성능으로 보고하면 두 배 가까이 부풀리는 셈임
- 두 모델의 무작위 CV는 0.639 와 0.629 로 사실상 같은데, 군집 CV는 0.373 과 0.183 으로 갈림 — 무작위 점수만 놓고 모델을 고르면 학습하지 않은 권역에서 더 많이 무너지는 쪽을 고를 수 있음

8.3 SHAP: 예측을 피처별 기여로 분해

□ 모델이 어느 열을 보고 예측했는지 확인하는 도구가 **SHAP**임(Lundberg & Lee, 2017). 개별 예측 하나를 피처별 기여값으로 분해함

- 논리는 몫 나누기임. 어떤 피처를 추가했을 때 예측이 얼마나 움직이는지를, 가능한 모든 추가 순서에 걸쳐 평균해 그 피처의 몫으로 정함

□ 예: 평균 예측 45점, 고령인구율만 반영하면 52점, 병원 거리만 반영하면 48점, 둘 다 반영하면 59점이라 하자

- 고령인구율을 먼저 넣으면 기여 = $52 - 45 = 7$ 점, 병원 거리 다음에 넣으면 기여 = $59 - 48 = 11$ 점
- 두 순서를 평균: 고령인구율의 SHAP = $(7 + 11) / 2 = 9$ 점. 같은 방식으로 병원 거리 = 5점
- $45 + 9 + 5 = 59$ 로 예측값이 정확히 복원됨. SHAP 값은 예측값과 같은 단위(여기서는 취약성 점수)를 가짐
- 같은 실행이 찍은 피처별 평균 절대 SHAP과 잔차 진단

출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log

[4.5 모델 해석] SHAP 피쳐 기여도 (RandomForest)

피쳐	평균 SHAP
elderly	4.003
dist_hosp	2.476
y	2.123
basic	1.365
dist_transit	1.354
single_hh	1.035
x	0.795
ndvi	0.456

[누수 점검] 잔차 Moran's I = 0.663 (p = 0.001)

주의: 잔차에 공간 구조가 남음 → 공간 피쳐/모델 보강 여지.

□ 화면에 나온 값에서 읽을 것

- 열 이름은 차례로 고령인구율·병원 거리·좌표 y·기초수급율·대중교통 거리·1인가구율·좌표 x·평균 NDVI임
- **좌표 y가 3위**임 — 5.5에서 예고한 신호가 실제로 나타난 자리. 취약성과 직접 관련 없는 위치 변수가 상위로 올라온 것은, 모델이 잠재 공간 구조를 좌표로 보간하고 있다는 뜻임. 8.2에서 무작위 검증(0.639)과 군집 검증(0.373)의 격차가 컸던 것과 같은 원인임
- 예측 잔차(실제-예측)의 Moran's I가 **0.663**(p=0.001)임 — 좌표를 넣었는데도 오차가 여전히 공간적으로 뭉쳐 있음. 좌표만으로는 잠재 구조를 다 담지 못했다는 뜻이며, 이럴 때는 점수 하나가 아니라 이 한계를 결과에 함께 적음

주의. SHAP이 보여주는 것은 모델이 무엇을 보고 판단했는가지 인과가 아님. 병원 거리의 SHAP이 크다는 사실에서 "병원을 지으면 취약성이 내려간다"는 결론이 따라오지 않음 — 모델은 병원 거리라는 열을 통해 도시화 수준 같은 다른 조건을 대리해 보고 있을 수 있음

8.4 순위표 읽기

- 같은 실행이 마지막에 찍는 우선지원 후보 목록

출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log

[정책] 취약성 예측 상위 5개 격자(우선지원 후보)

cell	elderly	basic	dist_hosp	pred	priority_rank
56	0.32	0.13	1575.86	59.40	1
16	0.30	0.06	1998.16	57.50	2
2	0.28	0.10	1507.67	56.66	3
35	0.26	0.10	1754.41	56.50	4
41	0.31	0.10	1693.43	54.96	5

전체 우선순위 저장 → ch4_vulnerability_priority.csv

○ 1위 59.40점과 5위 54.96점의 차이가 4.44점임. 400개 격자 전체의 순위는

results/ch4_vulnerability_priority.csv 에 저장됨

□ 순위표는 다음 네 가지를 순서대로 확인하며 읽음

1. **단순 기준과 비교함** — 모델 없이 고령인구율 상위 25%만 골랐을 때의 목록을 먼저 만들고, 모델의 순위가 그 목록과 어디서 달라지는지 봄. 모델의 기여는 이 기준선을 넘는 부분에서 나옴
2. **순위의 간격을 봄** — 1위와 5위의 차 4.44점은, 군집 검증 R^2 0.373이 뜻하는 오차 수준을 고려하면 순서를 확정할 만한 간격이 아님. 이럴 때는 1~5위를 개별 순위가 아니라 **상위 그룹**으로 묶어 처리함
3. **임계값 주변을 확인함** — 지원 기준이 상위 20개라면 20위와 21위 근처를 따로 뽑아 봄. 예측이 조금만 달라져도 대상 여부가 바뀌는 단위들임
4. **예상과 다른 결과를 그대로 보고함** — 고령인구율이 낮는데 상위에 오른 격자가 있으면 다른 피처의 기여를 확인함. 설명되면 유지하고, 설명되지 않으면 입력값 오류를 검토함. 결과를 본 뒤 예상을 고쳐 쓰지 않음

○ 네 번째 절차의 확인 대상이 위 표에 이미 있음 — 4위 격자(56.50점)의 고령인구율은 0.26으로 5위 격자(0.31)보다 낮음. 순위를 그대로 쓰기 전에 이 격자에서 무엇이 점수를 올렸는지 개별 SHAP으로 확인함

9. 핵심 요약

- **검증 방법이 점수를 바꿈.** 같은 자료·같은 모델에서 무작위 0.869, 블록 0.798, 군집 0.349. 어느 값을 쓸지는 재려는 능력 — 빈 곳 메우기인지 새 지역 예측인지 — 이 정함 (2절, 7절)
- **여덟 단계는 순서가 곧 근거임.** 앞 단계의 산출물이 뒷 단계의 입력이며, 검증 설계를 정하기 전에 모델 설정을 고르면 부풀려진 점수에 맞춰 최적화됨 (3절, 그림 4.1)
- **첫 결정은 표의 한 행.** 포인트·격자·행정구역·필지·네트워크 권역 중 무엇을 한 행으로 두는지가 자료·피처·전달 단위를 정함. 집계 단위를 바꾸면 결과가 바뀌는 MAUP는 없앨 수 없으므로, 현상 단위에서 예측하고 집계 단위로 가중평균해 모으며 가중치를 명시함 (4절)
- **공간 피처는 이웃 정의에서 시작함.** 공간가중행렬 W로 이웃을 정하고 행 표준화한 뒤, 이웃 값의 가중평균인 공간 시차를 열로 만들. 원변수와 시차의 차가 국지적 돌출을 드러냄 (5절)
- **누수는 두 종류이고 처방이 다름.** 목표값 누수(예측 시점에 없는 값이 열에 들어감)는 열을 빼서 고치고,

공간 분할 누수(인접 표본이 학습·검증에 나뉘어 들어감)는 나누는 방식을 바꿔 고침. 둘 다 오류 없이 통과하므로 사람이 확인해야 함 (5.5, 7.1)

○ **Moran's I가 검증 설계를 정함.** 실습 자료는 0.866($z=40.84$)으로 공간 분리 검증 필수 구간. 단, 0 근처라고 구조가 없다고 단정할 수 없음 — 4칸 예에서 두 덩어리가 붙어 있는데도 $I=0$ 이 나옴 (6절)

○ **공간 교차검증은 지역을 통째로 나눔.** 블록→버퍼→군집 순으로 분리가 강해지고, 점수는 낮아지며 폴드 간 변동은 커짐. 재려는 차이가 폴드 간 표준편차보다 작으면($0.071 < 0.143$) 그 차이를 결과로 인용하지 않음 (7절, 그림 4.2)

○ **결과 해석은 세 값을 함께 봄.** SHAP 순위에 좌표가 오르면(좌표 y 3위) 모델이 위치로 보간 중이라는 신호이고, 잔차 Moran's I가 높으면(0.663) 설명 못 한 공간 구조가 남았다는 뜻이며, 순위 간격이 오차 수준 안이면(1~5위 차 4.44점) 개별 순위가 아니라 상위 그룹으로 처리함. SHAP은 인과가 아님 (8절)

10. 과제

□ 이번 장의 과제는 **검증 점수 두 개의 차이를 직접 확인하고, 그 차이의 의미를 판단하는 것**

○ 명령 한 줄이면 제출할 파일이 생성됨

```
python scripts/submit.py 4 --id 학번 --name 이름
```

□ 이 명령이 대신 처리하는 작업: 실습 자료 생성(4-0-simdata-prep.py), 대표 실습 4-2-spatial-cv.py 실행, 제출용 파일 생성

○ 끝나면 submissions/ 폴더에 제출_4장_학번.md 파일이 생기고, 실행 결과가 이미 붙어 있음

○ **눈여겨볼 곳은 무작위 교차검증 점수와 공간 블록 교차검증 점수의 차이**

○ 맨 아래 빈칸 세 개를 채우면 됨

1. 눈에 띈 숫자 하나를 그대로 옮겨 적고, 왜 눈에 띄었는지 한 문장으로
2. 그 숫자가 왜 그렇게 나왔는가
3. 무작위 교차검증 점수를 그대로 보고서에 실었다면, 그 보고서는 무엇을 잘못 말하게 되는가?

□ **3번이 이 과제의 핵심.** 1·2번은 로그를 읽으면 답이 나오지만, 3번은 6~7절의 내용으로 판단해야 함

○ 채운 파일 **하나만** 올리면 제출이 끝남

- **막혀도 제출함**
- 실행이 도중에 실패하면 오류 메시지가 그 파일에 자동으로 담기고, 질문이 "어디서 막혔나"로 바뀜
- 어디서 왜 막혔는지 적은 제출도 온전한 제출. 실행 성공 여부로 점수를 매기지 않음

- **이 장은 학번을 난수 시드로 씀**
- 그래서 제출 파일의 숫자는 교재의 값과, 그리고 옆 사람의 값과 다른 것이 정상임

- 숫자는 달라도 무작위 검증이 블록 검증보다 높다는 방향은 같아야 함. 방향까지 다르면 그 이유를 찾는 것부터가 과제